

Leon Hamnett Peak Hour Demand Forecasting Report

Task summary:

The task is to **predict the probability that each hour of a day will be the peak load hour** using historical data on electricity load and weather.

Evaluation Metric

Predictions were evaluated using a **squared error scoring rule** for each hour:

$$S=(p-a)^2$$

where

- p = predicted probability
- a = actual outcome (1 if the hour is the peak hour, else 0)

The **total score** is the sum of all hourly scores across 2008.

Lower scores indicate better performance.

General methodology:

- Explore provided data
- Convert weather data into long format, per date+hour we have temp reading for all stations
- Combine long weather data with load table
- Explore combined data – looking for features which correlate or drive ‘Load’.
- Establish baseline error rate using simple heuristics
 - Find probability an hour would be peak load based on training subset. Apply fixed probabilities against validation subset, compare against true peak hours. Determine baseline error.
 - For each block of 24 hours, we can rank the hours from highest load to lowest load. By converting this ranking system into probabilities from training subset, and making predictions against validation subset – this gives us another baseline error to use.
- Engineer features – additional datetime features eg day of week, month, cyclical encoding, incorporate USA holiday data, additional statistics for temperature data
- Explore different modelling approaches related to classical ML. Look into possibility of making binary classification prediction per hour, also building regression model to predict load and then post-process into probabilities.
- As probabilities are required, this seems to lead towards deep learning model. Explore deep learning models and various combinations of input features.

- Finally decide most suitable approach is processing input features in a 24 hour block, with the target being a one-hot vector of 24 elements, with a 1 in the location of the peak load.

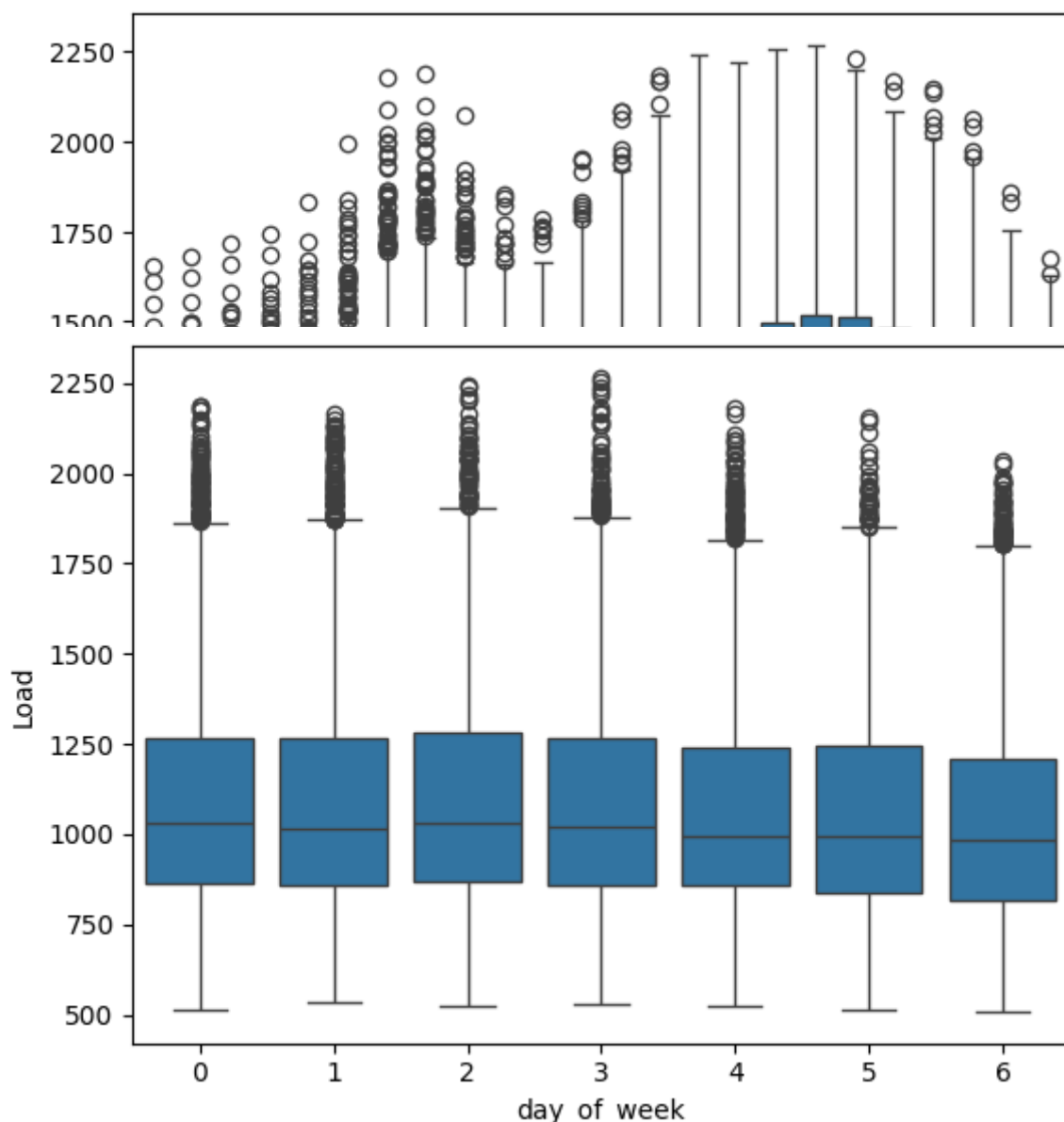
Datasets:

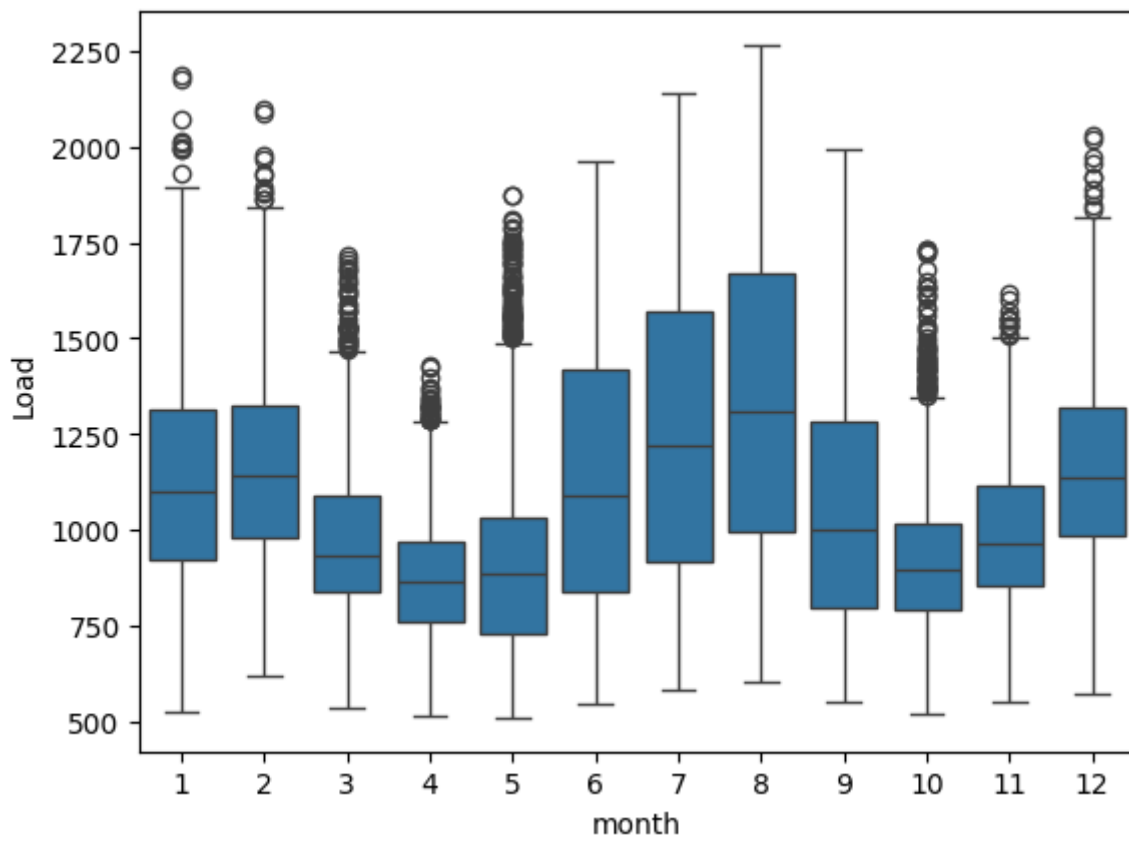
The load history dataset (load_hist_data.csv) contains 26,280 hourly observations spanning from 1 January 2005 to 31 December 2007. It includes three columns: Date, Hour, and Load, representing the total system electricity demand for each hour. The data are complete with no missing values. Load values range from 509 MW to 2,266 MW, with a mean of approximately 1,072 MW and a standard deviation of 314 MW, indicating moderate daily and seasonal variability typical of electricity demand patterns.

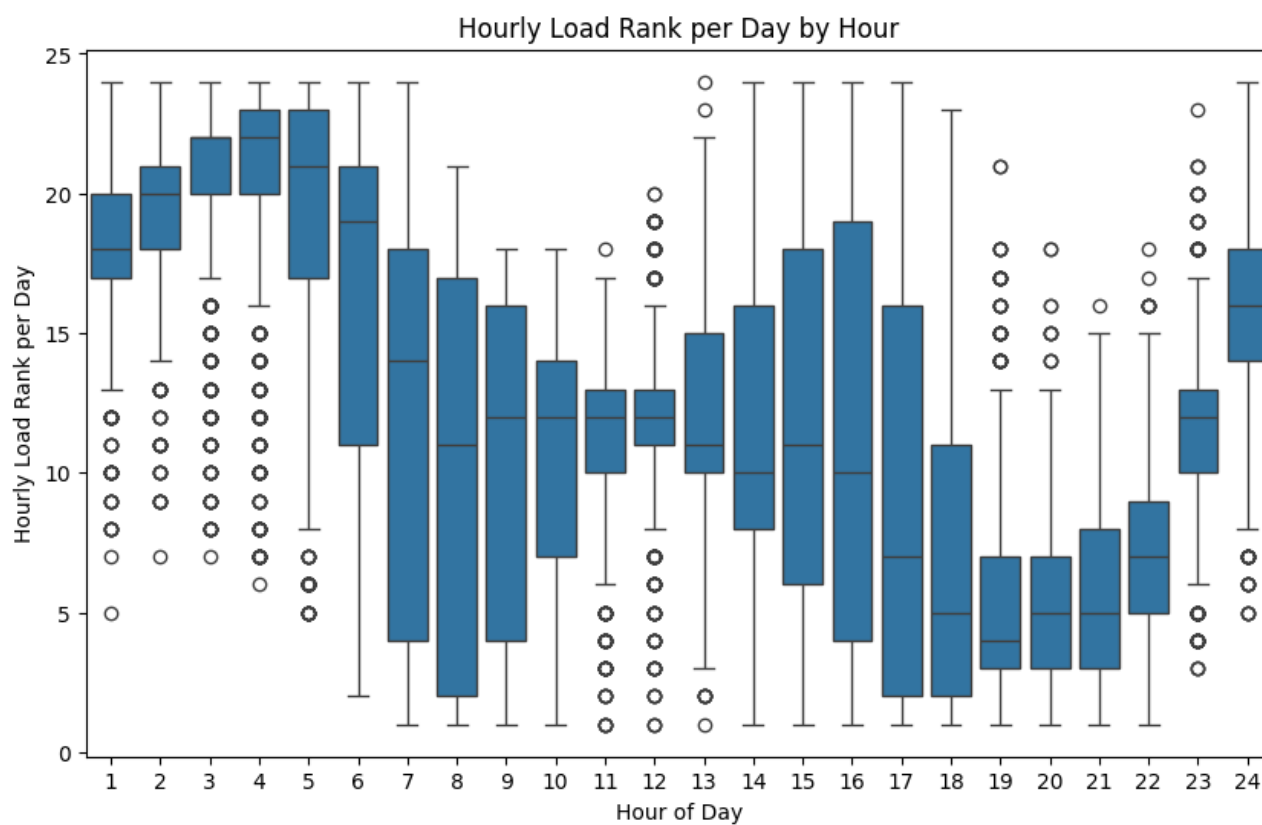
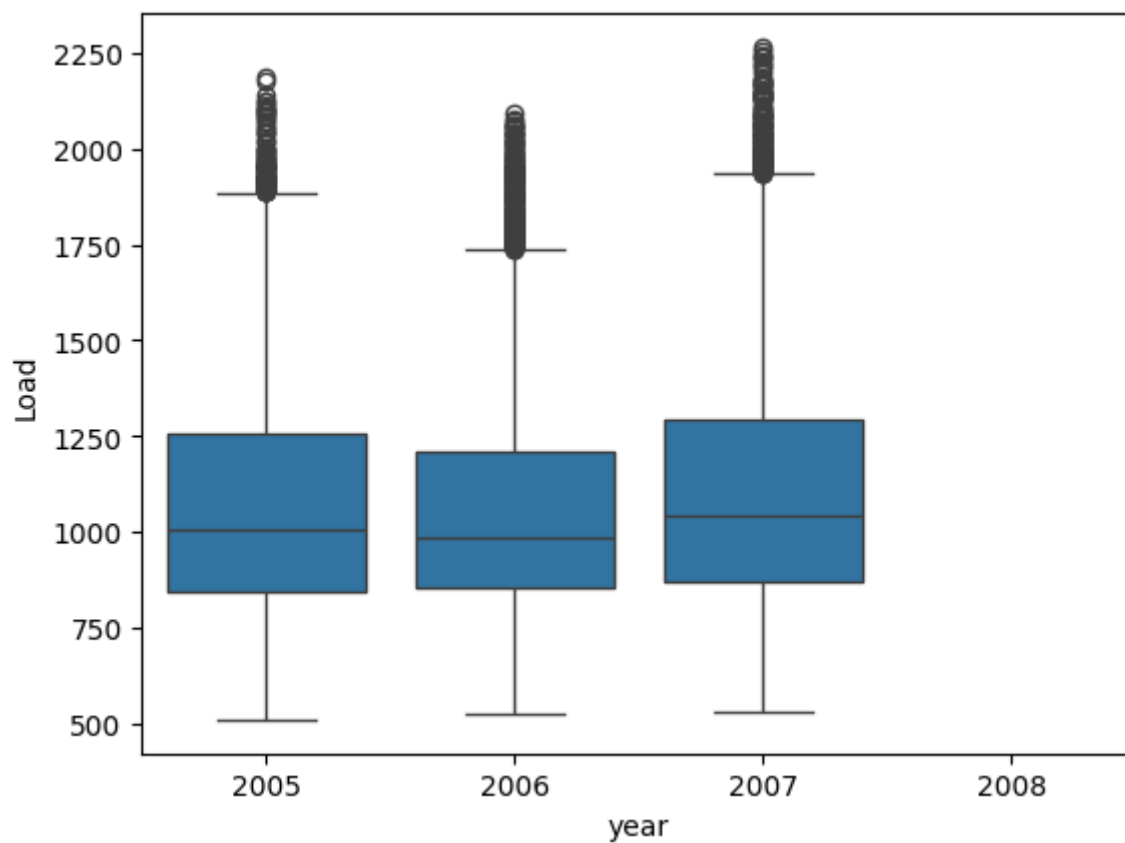
The weather dataset (weather_data.csv) consists of 981,792 hourly records collected from 28 weather stations between 1 January 2005 and 31 December 2008. Each record includes the date, hour, station ID, and temperature (°F). Every station provides 35,064 hourly entries, confirming full coverage with no missing data. Exploratory analysis using box plots revealed generally consistent temperature distributions across stations, though Station 9 displayed a noticeable number of outliers, suggesting potential sensor errors or localized climatic variation.

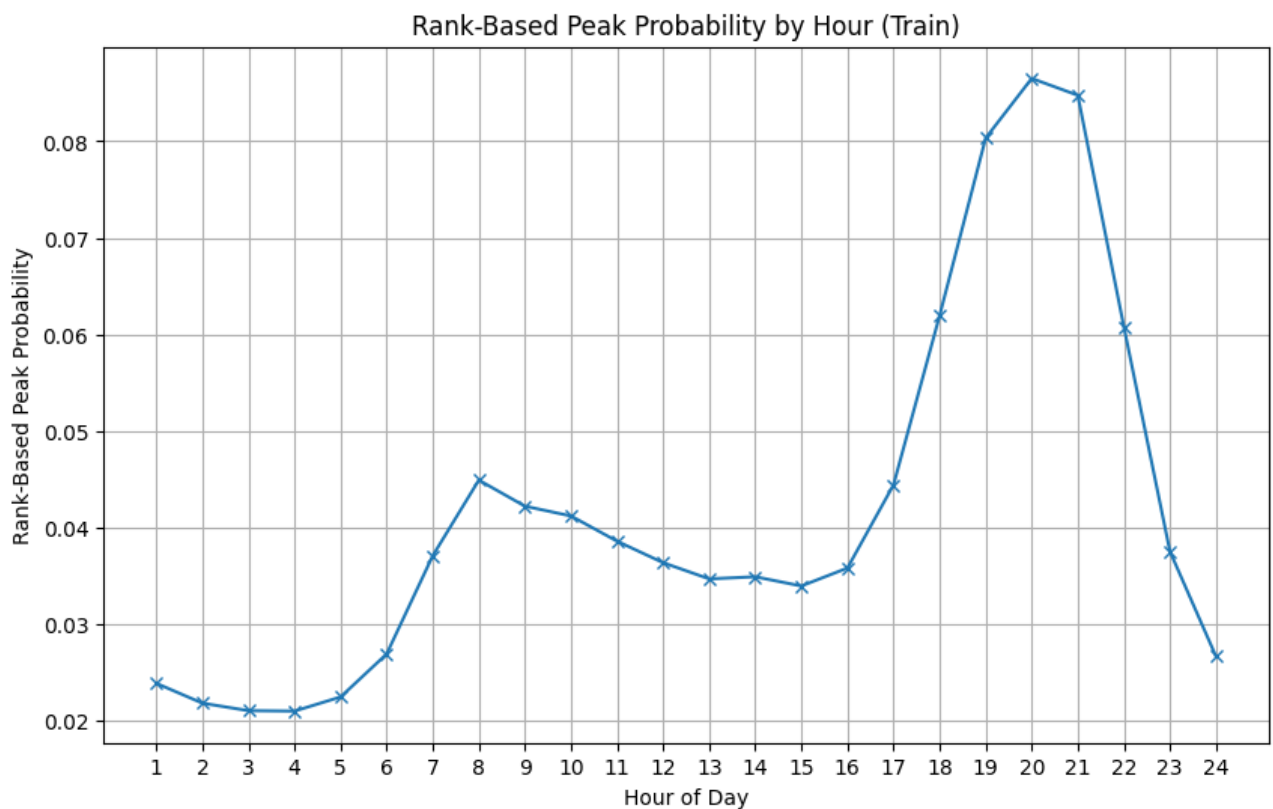
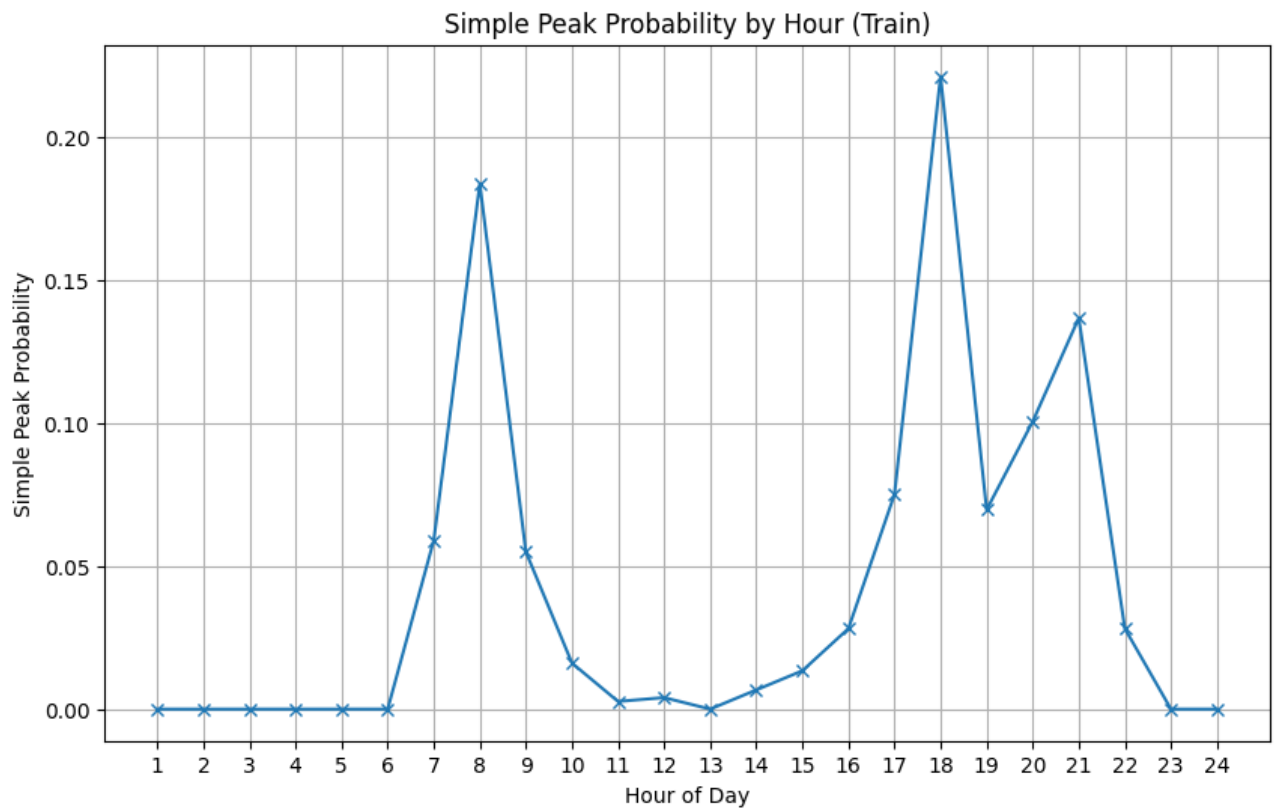
EDA:

As part of the exploratory data analysis process, we try to understand how the various time scales are impacting the Load value.









Data split:

Initially, the data were divided chronologically to preserve the temporal structure of the time series. The years **2005 and 2006** were used for **training**, allowing the model to learn typical load-weather

relationships and seasonal patterns. The year **2007** served as a **validation set**, used to tune features, evaluate model performance, and guide experimentation. Once the model's behaviour and feature set were satisfactory, the **training and validation data were combined** to maximise the amount of historical information available for final training. The year **2008** was held out as the **test set**, representing unseen future data for which only weather features were available—aligning with the task's real-world forecasting objective.

Feature engineering:

Feature engineering focused on capturing **temporal, weather, and holiday-related patterns** that influence electricity demand, while ensuring that all transformations were applied **without leaking information across training and test periods**. The process began by constructing a comprehensive set of **datetime features**, such as day of the week, month, and whether the hour fell on a weekend. To help models learn cyclical patterns, **sine and cosine encodings** were used for features like hour, day of week, and month—ensuring that, for example, hour 23 is numerically close to hour 0. A simple **season variable** (winter, spring, summer, autumn) was also introduced to capture broader seasonal trends in energy consumption.

Next, a suite of **holiday-based features** was developed using the `holidays` package. Each date was checked against the US federal holiday calendar to determine whether it was a holiday, what type of holiday it was (e.g., *winter_family*, *summer*, *government*, *thanksgiving*), and the number of days **to and since the nearest holiday**. These features are valuable because energy consumption patterns often shift around public holidays due to industrial shutdowns or changes in residential usage. Care was taken to map holidays by name and category while ensuring that feature creation relied only on information **available at the time**—avoiding any forward-looking bias.

A set of **prior probability features** was engineered to provide the model with a statistical anchor based on historical load patterns. The idea was to estimate the probability that a given hour—either by itself or in combination with the weekday—would be the **peak load hour** for that day. This was achieved by grouping the training data by the chosen granularity (e.g. hour or hour-weekday pair) and calculating the average frequency with which each combination corresponded to the daily peak. These frequencies were then **normalised** to form a probability distribution and stored as a mapping dictionary.

When applied to new data, each record inherited its corresponding prior probability from the training data on the hour or hour-weekday combination. The intuition behind this approach was that these **priors act as a baseline behavioural pattern** for the model—essentially encoding “what usually happens” in the data. By providing this historical anchor, the model could more easily learn **contextual deviations** caused by weather, holidays, or other dynamic factors, rather than having to infer these long-term temporal tendencies from scratch.

Finally, weather data were summarised through **aggregate temperature statistics across all 28 stations** for each hour, including the maximum, minimum, mean, standard deviation, and range (temperature difference). These summary statistics capture both overall climate conditions and geographical variability, which can influence regional demand levels. Categorical variables such as `season`, `holiday_type`, and `holiday_name` were **one-hot encoded** to make them usable in machine learning models. Together, these engineered features provided a structured, interpretable, and leakage-safe input space for downstream model training and evaluation.

Insights

There are a number of days which have the identical load for several hours:

['2005-01-03', '2005-05-10', '2005-08-05', '2005-09-04', '2005-10-02', '2005-10-16', '2005-12-07', '2006-01-25', '2006-06-05', '2006-08-04', '2006-09-01', '2006-10-03', '2006-11-02', '2006-11-09', '2006-12-07', '2006-12-23', '2007-02-27', '2007-04-07', '2007-05-06', '2007-06-09', '2007-10-01', '2007-11-16', '2007-12-10']

To prevent the case where we end up with 2 peak hours and invalidating the initial conditions that each day has a single peak, we chose one peak hour randomly for each of these cases.

Modelling experimentation approach:

The modelling process began with a set of **simple probabilistic heuristics** to establish a baseline understanding of the task. The first heuristic estimated the **probability that a given hour is the peak load hour of the day**, based purely on historical data. This was calculated by grouping all records by hour, computing the mean frequency of that hour being the peak, and then **normalising these probabilities** across the 24-hour period. A second, related heuristic used each hour's **average rank** (from 1 to 24, where 1 is the highest load) as an inverse measure of its likelihood to be the daily peak. These baselines were evaluated against the validation set to generate initial error estimates, providing a reference for model improvement.

Building on these heuristics, the next stage introduced **supervised learning models**. First, **binary classifiers** were explored to gain understanding of the day. These binary classifiers aimed to predict whether each hour represented the daily peak based on engineered features such as temperature, calendar effects, and time of day.

Then, a **regression-based approach** was explored, predicting hourly load directly and **post-processing** predictions by identifying the hour with the maximum forecasted value as the predicted peak. To make these regression outputs comparable, the 24 predicted hourly loads were passed through a **softmax transformation** to yield a probability distribution over the hours.

Finally, the project transitioned to a **deep learning formulation**, where the model processed data in **24-hour blocks** representing each day's full feature set. The target was a **one-hot encoded vector of length 24**, where a single "1" indicated the true peak hour.

This allowed the network to learn temporal dependencies and relative relationships between hours. Throughout development, models were trained and tuned using the **validation set (2007)**, and once satisfactory performance was reached, **training and validation data were combined** for a final model trained on all historical years and tested on **2008**, simulating a realistic forward-prediction scenario.

Modelling Results:

To establish a performance baseline, a **simple probabilistic heuristic** was first implemented. This model estimated the likelihood of each hour being the daily peak based purely on historical frequencies from the training data. When evaluated against the **2007 validation set**, this baseline produced an **error rate of 323.37**, serving as the initial benchmark to improve upon. A variant of

this approach, which used the **average historical rank** of each hour (with lower ranks corresponding to higher loads), was also tested but performed slightly worse, yielding an **error rate of 347.47**.

Subsequent experiments explored a range of **machine learning formulations**. Hour-level **binary classification models** were trained to predict whether each hour represented the daily peak, while **regression models** aimed to forecast load values directly. The regression outputs were then **normalised with a softmax function** to convert them into per-day probabilities of being the peak hour. However, these conventional models struggled to outperform the initial heuristic, suggesting that hour-by-hour learning was insufficient to capture daily temporal dependencies.

A significant improvement came from reframing the task as a **deep learning problem using daily 24-hour blocks**. The first deep model, using only raw temperature data from weather stations and hourly priors (without feature scaling), achieved an **error rate of 314.23**. Incorporating a **standard scaler** and **summary statistics across stations** (mean, max, min, and variance) reduced the error further to **302.16**. After adding more comprehensive **feature engineering**, including calendar and holiday effects, the **basic deep learning model achieved an error rate of 250 on the validation set**—a substantial improvement over all previous approaches.

Closing thoughts

While the results demonstrate strong progress, there are several avenues for refinement and deeper experimentation.

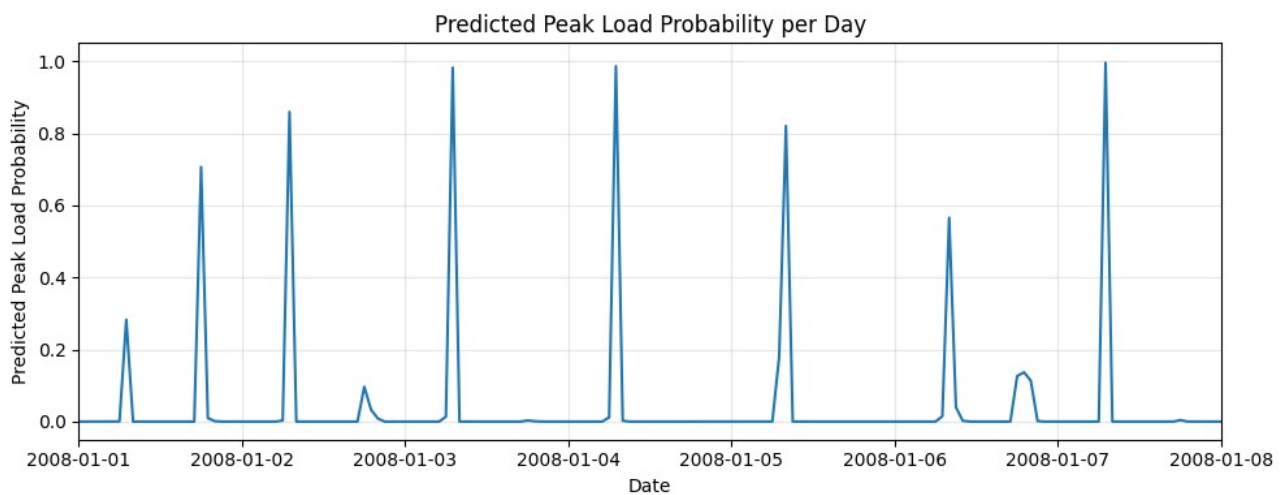
Firstly, the **input feature set could be cleaned and optimised** further. When hourly data are aggregated into daily blocks, features such as `day_of_week` and `month` are repeated across all 24 hours, which adds unnecessary redundancy. A more systematic feature engineering approach: such as separating **day-level and hour-level features** - would make the model's representation more efficient and interpretable. Similarly, feature selection or dimensionality reduction techniques (e.g. PCA or feature importance analysis) could help remove redundant signals while retaining key temporal and weather-related variations.

The **deep learning architecture** used in this study was intentionally simple to establish a clear baseline. However, more sophisticated architectures could capture the sequential and contextual nature of hourly load patterns more effectively. For instance, **recurrent neural networks (RNNs)** or LSTMs, or **temporal convolutional networks (TCNs)** could better model intra-day dependencies, while **attention-based models or transformers** could learn relationships across longer time spans or non-linear holiday-weather interactions. Systematic **hyperparameter tuning** including adjustments to learning rates, hidden units, dropout rates, and activation functions could also lead to measurable gains in performance.

By encoding the output as a one-hot vector, the model is effectively being trained to identify a single dominant peak per day. This may limit its ability to capture more complex load patterns, such as days with multiple peaks (e.g., morning and evening). An alternative formulation would be to represent the **target as a vector of load proportions across the 24 hours of a day**. This approach would encourage the model to learn a smoother probability distribution, potentially improving its ability to represent multi-peak behaviour.

In future iterations, aspects of this workflow could be further streamlined using MLOps practices. Automating data preparation, model training, and evaluation pipelines would allow faster experimentation and easier tracking of model performance over time.

Beyond architecture, there are promising directions for **model ensembling and hybrid approaches**. The output of a regression model predicting hourly load values could be incorporated as an additional input feature, providing a grounded estimate of magnitude alongside the classification objective of identifying the peak hour.



As shown in the graph above, the softmax predictions give a marked peak per day. It could be beneficial to tune the **temperature parameter of the softmax layer** to control the sharpness of the output distribution allowing for a smoother probability spread over likely peak hours rather than forcing a single dominant prediction. This would yield better-calibrated probabilities and might improve interpretability for operational decision-making. Together, these improvements would create a more robust and context-aware system capable of adapting to subtle changes in demand dynamics over time.